



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации информационных
технологий

ОТЧЕТ ПО ПРОЕКТУ

по дисциплине «Системная и программная инженерия»

Студент группы ИКБО-50-23

Павлов Н.С.

(подпись студента)

Руководитель проектной работы

Мельников Д.А.

(подпись руководителя)

Работа представлена

«___» _____ 2026 г.

Допущен к работе

«___» _____ 2026 г.

Москва 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	2
1. ПЛАНИРОВАНИЕ И ПОДГОТОВКА	4
1.1. Целевая аудитория	4
1.1.1. Основной портрет	4
1.1.2. Второстепенный портрет	4
1.2. Требования к системе	4
1.2.1. User Story	4
1.2.2. Функциональные требования	5
1.2.3. Нефункциональные требования	6
1.2.4. Матрица требований	2
1.3. Техническое задание.....	2
1.4. План реализации проекта	3
2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ	4
2.1. Поведенческие диаграммы	4
2.1.1. Диаграмма вариантов использования	4
2.1.1.1. Обоснование сценариев использования.....	4
2.1.1.2. Схематическое представление (Use Case Diagram).....	4
2.1.2. Диаграмма последовательности	5
2.2. Процесс работы в системе	6
2.3. Структурные диаграммы	7
2.3.1. Диаграмма классов.....	7
2.3.2. Диаграмма объектов.....	8
2.4. Информационное взаимодействие компонентов системы	8
2.4.1. Текстовое описание.....	8
2.4.2. Диаграмма DFD.....	9
2.5. Модель Базы данных	13
2.6. Архитектура приложения	14
2.6.1. Обоснование выбора	15
2.6.2. Диаграмма архитектуры	17

ВВЕДЕНИЕ

Современная жизнь требует от человека, особенно от студента, грамотного управления личными финансами. Проект "Finlytics: Офлайн-менеджер финансов" на языке Kotlin направлен на разработку десктопного приложения, которое предоставит пользователю удобный инструмент для учета доходов и расходов, категоризации трат и визуализации финансовой статистики без необходимости подключения к интернету. Основной целью является создание простого, интуитивно понятного и производительного приложения, которое помогает пользователям контролировать свой бюджет.

Краткое описание проекта:

Проект представляет собой десктопное приложение "Finlytics" для управления персональными финансами. Ключевые функции: внесение операций "доход" и "расход" с возможностью их категоризации, фильтрация данных по различным временным промежуткам, а также наглядное представление финансовой статистики в виде диаграмм и списков.

Проблематика:

Актуальность проекта обусловлена необходимостью простого и доступного инструмента для планирования и контроля личного бюджета. Многие существующие решения либо требуют постоянного подключения к интернету, либо обладают избыточной сложностью, либо неудобны в повседневном использовании. Студенты, как социальная группа с часто ограниченным бюджетом, особенно нуждаются в таком инструменте для формирования навыков финансовой грамотности.

Для достижения цели необходимо выполнить следующие задачи:

- Задача 1: Разработать архитектуру данных и пользовательский интерфейс для удобного внесения и редактирования финансовых операций.

- Задача 2: реализовать механизм категоризации доходов и расходов с возможностью добавления пользовательских категорий.
- Задача 3: внедрить систему фильтрации и анализа данных по временным промежуткам (день, неделя, месяц, год, все время) с визуализацией результатов в виде диаграмм и отчетов.

Состав команды и распределение ролей:

- Враженко Д.О. – Team Lead, менеджер проекта, разработчик, тестировщик, технический писатель. Ответственный за постановку задач, общую архитектуру проекта, разработку ключевых модулей и составление отчетности.
- Хохряков А.Ю. – Аналитик, разработчик, тестировщик, технический писатель. Ответственный за реализацию модуля работы с данными и бизнес-логики.
- Павлов Н.С. – UI/UX дизайнер, разработчик, тестировщик, технический писатель. Ответственный за проектирование и реализацию пользовательского интерфейса, включая визуализацию данных (диаграммы).

1. ПЛАНИРОВАНИЕ И ПОДГОТОВКА

1.1. Целевая аудитория

1.1.1. Основной портрет

Студент, начинающий контролировать личный бюджет.

Это основная целевая аудитория. Данный пользователь — молодой человек или девушка, часто впервые сталкивающийся с необходимостью самостоятельного управления финансами (стипендия, подработки, деньги от родителей). Его бюджет ограничен, и он ищет простой и понятный инструмент, чтобы отслеживать, на что уходят деньги, и учиться планировать траты.

1.1.2. Второстепенный портрет

Занятый профессионал, ценящий приватность и офлайн-доступ.

Этот пользователь — работающий человек, который уже имеет некоторый финансовый опыт, но предпочитает держать свои данные под полным контролем. Он ценит скорость и эффективность, у него нет времени на изучение сложного софта. Ему нужен надежный инструмент для быстрой фиксации основных операций и получения сводки без подключения к интернету.

1.2. Требования к системе

1.2.1. User Story

Таблица 1.1 – User Story

Кто?	Что хочет?	С какой целью?
Пользователь	Чтобы при первом запуске в приложении уже был предустановленный набор категорий	Чтобы не приходилось тратить время на их создание с нуля, и пользователь мог сразу начать вносить свои траты
Пользователь	Быстро добавить информацию о том, что потратил деньги, указав сумму,	Чтобы точно фиксировать все траты

	категорию и дату	
Пользователь	Внести запись о получении денег	Чтобы видеть полную картину своих финансов
Пользователь	Иметь возможность создавать свои собственные категории для доходов и расходов	Чтобы не ограничиваться предустановленными категориями
Пользователь	Когда открывается главный экран, отображать, какой баланс, а также общую сумму доходов и расходов за выбранный период	Чтобы быстро оценить свое финансовое состояние
Пользователь	Видеть наглядную диаграмму, которая показывает, на что именно тратятся деньги больше всего	Чтобы понимать структуру своих расходов
Пользователь	Иметь возможность посмотреть свою финансовую статистику и историю операций не только за всё время, но и за конкретный период (день, неделю, месяц или год)	Чтобы отслеживать динамику трат и доходов
Пользователь	Иметь возможность открыть запись и редактировать её	Чтобы устранить ошибку при записи данных
Пользователь	Иметь возможность удалить запись из истории	Чтобы она не учитывалась в общей статистике
Пользователь	Иметь возможность удалять созданные мной категории	Чтобы убрать неиспользуемые категории
Пользователь	Иметь возможность сравнивать свои текущие показатели трат с предыдущим месяцем	Чтобы понимать тенденцию и частоту своих трат и корректировать свои планы для большей экономии.

1.2.2. Функциональные требования

На основе описанных User Story были сформулированы следующие функциональные требования проекта:

- Система должна автоматически создавать предустановленный список категорий доходов и расходов при первом запуске приложения (инициализация БД).
- Пользователь должен иметь возможность создавать новые категории доходов и расходов.
- Пользователь должен иметь возможность удалять существующие категории.
- Система должна блокировать удаление категории, если она используется хотя бы в одной операции.

- Пользователь должен иметь возможность добавлять операции типа "Доход" и "Расход".
- Операция должна содержать: сумму, категорию, дату и описание (необязательно).
- Система должна валидировать вводимые данные: сумма должна быть положительным числом, дата — корректной, категория — выбрана из списка.
- Пользователь должен иметь возможность редактировать любую существующую операцию.
- Пользователь должен иметь возможность удалять любую существующую операцию.
- На главном экране отображать сводку: баланс, общие доходы и расходы за выбранный период.
- На главном экране отображать круговую диаграмму распределения операций по категориям (с переключением "Доходы/Расходы").
- На экране истории отображать полный список операций.
- Реализовать фильтрацию данных на главном экране и в истории по временным промежуткам (день, неделя, месяц, год, все время).
- Реализовать фильтрацию в истории по типу операции (Все/Доходы/Расходы).
- Реализовать timeline с моделью «Призрака» с показателями трат за текущий месяц в сравнении с прошлым месяцем.

1.2.3. Нефункциональные требования

Учитывая, что проект представляет собой десктопное офлайн-приложение, были определены следующие нефункциональные требования:

Таблица 1.2 – Нефункциональные требования

Тип требования	Содержание требования
Технические ограничения	Десктопное приложение должно функционировать на операционных системах семейства Windows (10 и выше), macOS

	(11 и выше) и Linux (дистрибутивы с поддержкой JRE).
	Приложение должно распространяться в виде исполняемого JAR-файла, для работы которого требуется Java Runtime Environment (JRE) версии 8 или выше.
	Для хранения данных должно использоваться исключительно локальное хранилище (SQLite) без необходимости подключения к интернету.
Производительность	Время отклика интерфейса на действия пользователя (открытие экрана, добавление операции, применение фильтра) не должно превышать 500 миллисекунд.
	Запуск приложения и загрузка начальных данных (история операций и категории) должны занимать не более 3 секунд.
	Операции фильтрации списка транзакций и пересчета статистики по временным периодам должны выполняться мгновенно, без заметных задержек.
Надежность	Приложение должно корректно завершать работу и сохранять целостность данных при закрытии окна или выключении компьютера.
	Сбои в работе отдельных компонентов (например, при попытке удалить используемую категорию) не должны приводить к полной остановке или некорректному состоянию приложения.
	Так как приложение является однопользовательским и однопоточным (в рамках одного окна), но для надежности, при любых операциях с БД должен использоваться корректный механизм блокировок, предоставляемый SQLite, чтобы предотвратить повреждение данных при гипотетических попытках записи из нескольких потоков.
Безопасность	Локальная база данных (SQLite) не должна быть доступна для чтения или изменения другими приложениями напрямую (защита на уровне файловой системы).
	Приложение должно блокировать SQL-инъекции при формировании запросов к базе данных (за счет использования параметризованных запросов в коде).
	Приложение не должно требовать прав администратора (root) для своей работы, чтобы минимизировать риски, связанные с повышением привилегий. Все операции с файлами должны выполняться в контексте прав текущего пользователя.
Удобство использования	Интерфейс приложения должен быть интуитивно понятным и не требовать от пользователя изучения дополнительной документации для выполнения основных сценариев (добавление, редактирование, удаление).
	Все ключевые элементы управления (кнопки, поля ввода) должны быть снабжены всплывающими подсказками (ToolTips), поясняющими их назначение.
	Система должна предоставлять пользователю понятные сообщения об ошибках ввода данных (например, при вводе некорректной суммы или даты).

1.2.4. Матрица требований

Матрица объединяет функциональные требования, выявленные на этапе планирования, и нефункциональные требования, сформулированные выше. Она структурирована по основным компонентам приложения и источникам.

Таблица 1.3 – Матрица требований

№	Требование	Суть	Автор	Ссылки	Критерии проверки	Компоненты архитектуры
1	Клиентское приложение (Desktop)					
1.1	Управление операциями					
1.1.1	Добавление операции	Пользователь должен иметь возможность добавить новую финансовую операцию с указанием типа (доход/расход), суммы, категории, даты и описания.	Павлов Н.С.	User Story (Ошибка! Источник ссылки не найден.)	При нажатии на кнопку "Добавить" открывается диалоговое окно, в котором можно ввести все параметры и сохранить операцию.	UI-компоненты (диалог добавления операции) ViewModel (обработка события сохранения) Репозиторий (координация записи) Слой данных (запись в таблицы транзакций)
1.1.2	Редактирование операции	Пользователь должен иметь возможность открыть существующую запись и изменить любые её параметры.	Павлов Н.С.	User Story (Ошибка! Источник ссылки не найден.)	При нажатии на кнопку "Редактировать" у элемента списка открывается диалог с пред заполненными полями, после изменения и сохранения данные обновляются.	UI-компоненты (список операций, диалог редактирования) ViewModel (хранение редактируемой операции) Репозиторий (координация обновления) Слой данных (обновление записей)
1.1.3	Удаление операции	Пользователь должен иметь возможность удалить запись из	Павлов Н.С.	User Story (Ошибка! Источник ссылки не найден.)	При нажатии на кнопку "Удалить" появляется диалог подтверждения.	UI-компоненты (диалог подтверждения)

		истории, при этом она перестает учитываться в статистике.		не найден.)	После подтверждения операция исчезает из списка и не влияет на баланс и диаграммы.	ViewModel (обработка события удаления) Репозиторий (координация удаления) Слой данных (удаление записей)
1.2	Управление категориями					
1.2.1	Предустановленные категории	При первом запуске в приложении должен быть предустановленный набор категорий для доходов и расходов.	Хохряков А.Ю.	User Story (Ошибка! Источник ссылки не найден.)	После чистой установки и первого запуска в настройках отображаются категории: Зарплата, Стипендия, Еда, Транспорт и др.	Слой данных (инициализатор БД) SQLite (таблицы категорий)
1.2.2	Создание пользовательских категорий	Пользователь должен иметь возможность создавать свои собственные категории для доходов и расходов.	Хохряков А.Ю.	User Story (Ошибка! Источник ссылки не найден.)	В экране "Настройки" при нажатии на кнопку "+" появляется диалог, позволяющий ввести имя новой категории и добавить её в соответствующий список.	UI-компоненты (диалог добавления категории) ViewModel (проверка уникальности) Репозиторий (координация добавления) Слой данных (запись в таблицы категорий)
1.2.3	Удаление категорий	Пользователь должен иметь возможность удалять созданные им категории. Система должна блокировать удаление категории, если она используется в операциях.	Хохряков А.Ю.	User Story (Ошибка! Источник ссылки не найден.)	При попытке удалить категорию, которая привязана к какой-либо операции, должно появляться предупреждение, и удаление не должно выполняться.	UI-компоненты (кнопка удаления) ViewModel (проверка наличия связанных операций) Репозиторий (координация удаления) Слой данных (удаление с проверкой внешних ключей)
1.3	Аналитика и просмотр					

1.3.1	Финансовая сводка	На главном экране должна отображаться сводка: баланс, общая сумма доходов и расходов за выбранный период.	Враженко Д.О.	User Story (Ошибка! Источник ссылки не найден.)	На экране "Обзор" при выборе периода (День/Неделя/Месяц) цифры в блоках "Доходы", "Расходы" и "Баланс" пересчитываются.	UI-компоненты (блоки статистики) ViewModel (расчет сводных показателей) Репозиторий (предоставление данных) Слой данных (извлечение транзакций)
1.3.2	Визуализация данных	На главном экране должна отображаться круговая диаграмма, показывающая структуру доходов или расходов по категориям за выбранный период.	Враженко Д.О.	User Story (Ошибка! Источник ссылки не найден.)	При переключении между режимами "Доходы" и "Расходы" на главном экране меняется диаграмма и список категорий, отображая соответствующие данные.	UI-компоненты (кастомный компонент диаграммы) ViewModel (группировка данных по категориям) Репозиторий (предоставление данных) Слой данных (извлечение транзакций)
1.3.3	Фильтрация истории	На экране истории пользователь должен иметь возможность фильтровать список операций по временному промежутку (день, неделя, месяц, год, все время) и по типу операции (все, доходы, расходы).	Враженко Д.О.	User Story (Ошибка! Источник ссылки не найден.)	При изменении параметров фильтрации на экране "История" список операций должен обновляться в соответствии с выбранными критериями.	UI-компоненты (элементы фильтрации) ViewModel (применение фильтров) Репозиторий (предоставление данных) Слой данных (извлечение транзакций)
2	Правовые нормы и стандарты					
2.1	Защита персональных	Локальная база	Враженко	ГОСТ 34.602-2020,	При проверке файловой	SQLite (файл базы

	данных	данных с персональными финансовыми данными пользователя не должна быть доступна для чтения сторонними приложениями.	Д.О.	Федеральный закон № 152-ФЗ "О персональных данных"	системы, файл базы данных (Finlytics.db) не должен открываться стандартными средствами ОС или другими программами без специального доступа.	данных) ОС (права доступа к файлам)
2.2	Соответствие стандартам документирования	Исходный код должен быть документирован в соответствии со стандартом KDос для языка Kotlin.	Враженко Д.О.	ГОСТ 34.201-2020, официальная документация Kotlin	При проверке исходного кода, все публичные классы и функции должны содержать KDос-комментарии с описанием.	Все компоненты системы (документирование кода)
3	Технические и нефункциональные ограничения					
3.1	Кроссплатформенность	Приложение должно работать в ОС Windows, macOS и Linux.	Враженко Д.О.	Анализ рынка аналоговых приложений	Собранный JAR-файл успешно запускается и корректно отображает интерфейс, а также выполняет все функции на каждой из трех целевых ОС.	Kotlin/JVM (кроссплатформенная среда) Compose for Desktop (кроссплатформенный UI) JRE (среда выполнения)
3.2	Производительность фильтрации	Операции фильтрации данных по временным промежуткам должны выполняться мгновенно, без заметных задержек для пользователя.	Павлов Н.С.	Нефункциональные требования (п. Ошибка! Источник ссылки не найден.)	При переключении между вкладками "День/Неделя/Месяц" на экране "Обзор" или "История" интерфейс обновляется быстрее, чем за 0.5 секунды.	ViewModel (оптимизированная обработка) Репозиторий (эффективное предоставление данных) SQLite (оптимизированные запросы)
3.3	Безопасность данных	Приложение должно использовать	Хохряков А.Ю.	Нефункциональные требования (п.	Проверка кода: все обращения к БД в	Слой данных (параметризованные

		параметризованные SQL-запросы для исключения возможности повреждения БД некорректным вводом.		Ошибка! Источник ссылки не найден.)	объектах AddCategory, AddIncomeTransaction и др. должны использовать PreparedStatement с плейсхолдерами (?).	запросы через JDBC)
--	--	--	--	--	--	---------------------

1.3. Техническое задание

При разработке технического задания (ТЗ) на создание программного продукта «Finlytics: Офлайн-менеджер финансов» в качестве основного руководящего документа был выбран межгосударственный стандарт ГОСТ 34.602-2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

Выбор данного стандарта обусловлен следующими факторами:

1. Целевая направленность. ГОСТ 34.602-2020 входит в комплекс стандартов на автоматизированные системы (АС) и специально разработан для регламентации процесса создания технического задания. Он устанавливает четкий состав, содержание и правила оформления документа, что позволяет структурировать требования к разрабатываемому продукту на самом раннем этапе его жизненного цикла.

2. Универсальность и полнота. Несмотря на то, что стандарт разработан достаточно давно, он носит универсальный характер и не привязан к конкретным технологиям или архитектурам. Структура ТЗ, предлагаемая стандартом, в полной мере подходит для описания широкого класса систем, включая современные десктопные приложения, такие как Finlytics. Это позволило всесторонне описать проект, не упустив ключевых аспектов: от целевой аудитории до технических требований.

3. Соответствие специфике проекта (Анализ применимости разделов). Все ключевые разделы стандарта были успешно адаптированы под особенности разрабатываемого офлайн-приложения.

4. Актуальность редакции. Версия стандарта 2020 года является действующей и актуальной. Использование устаревших редакций могло бы привести к несоответствию современным требованиям к оформлению и содержанию проектной документации.

Составленное ТЗ на разработку проекта представлено в файле ниже:

1.4. План реализации проекта

После детального анализа и выявления требований к системе был составлен план работ и распределены задачи и обязанности между участниками проекта в сервисе Miro (Рисунок 1.1).

	Title	Description	Status	Person	Start Date	End Date	
1	Выбор темы	Выбор темы проекта, определение основных идей. Распределение ролей внутри проекта	Complete	Даниил n1k_pavlov Andrey...	Feb 10, 2026	Feb 10, 2026	
2	Выявление требований	Выявление функциональных и нефункциональных требований. Построение диаграммы вариантов использования и диаграммы последовательности. Создание матрицы требований	Complete	Andrey... Даниил	Feb 10, 2026	Feb 24, 2026	
3	Проектирование сиситемы	Проектирование логики работы системы в различных нотациях (IDEFO, DFD, UML...)	Not Started	Andrey... n1k_pavlov			
4	Разработка ТЗ	Оформление итогового ТЗ, включающего все требования и модели	In Progress	Даниил	Feb 17, 2026		
5	Создание прототипа интерфейса	Создание макетов экранов в Figma, Разработка дизайн-системы (цветовая палитра, типографика, отступы). Подготовка кликабельного прототипа для демонстрации пользовательского пути	In Progress	n1k_pavlov	Feb 15, 2026		
6	Разработка модуля работы с БД	Разработка модулей для подключения к SQLite и инициализации базы данных. Реализация операций CRUD для категорий и транзакций. Формирование единого репозитория для доступа к данным	Not Started	Andrey...			
7	Разработка пользовательского интерфейса	Верстка главного экрана (обзорная статистика, фильтры, диаграмма). Верстка экрана истории операций (список, фильтрация). Верстка экрана настроек (управление категориями). Реализация диалогов для добавления/редактирования операций и категорий. Разработка настомной диаграммы для визуализации	Not Started	n1k_pavlov			
8	Связь модулей согласно MVVM	Написание базовой логики ViewModel (управление состоянием, обработка действий пользователя). Интеграция слоя данных с ViewModel. Подключение интерфейса к ViewModel (реактивное обновление данных). Реализация навигации между экранами. Интеграция диалоговых окон	Not Started	Даниил			
9	Тестирование	Тестирования асех сценариев использования. Тестирование корректности работы с базой данных (целостность, запросы). Тестирование пользовательского интерфейса на соответствие макетам	Not Started	Даниил n1k_pavlov Andrey...			
10	Подготовка отчетной документации	Документирование ключевых компонентов кода. Разработка руководства пользователя и прочей отчетной документации	Not Started	Даниил n1k_pavlov Andrey...			
11	Подготовка к презентации продукта	Демонстрация дизайн-решений и реализованного интерфейса. Проверка работоспособности приложения на целевых платформах. Презентация готового продукта	Not Started	Даниил n1k_pavlov Andrey...			

Рисунок 1.1 — Kanban-доска проекта

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1. Поведенческие диаграммы

2.1.1. Диаграмма вариантов использования

2.1.1.1. Обоснование сценариев использования

- Пользователь добавляет новую операцию (доход/расход).
- Пользователь редактирует существующую операцию.
- Пользователь удаляет операцию.
- Пользователь просматривает историю операций.
- Пользователь фильтрует операции по дате и категории.
- Пользователь просматривает финансовую сводку.
- Пользователь управляет категориями.

2.1.1.2. Схематическое представление (Use Case Diagram)

Диаграмма включает одного основного актора – Пользователь. Сценарии использования: "Добавить операцию", "Редактировать операцию", "Удалить операцию", "Просмотреть историю операций", "Отфильтровать операции", "Просмотреть финансовую сводку", "Добавить категорию".

Результат создания диаграммы представлен на Рисунок 2.1.

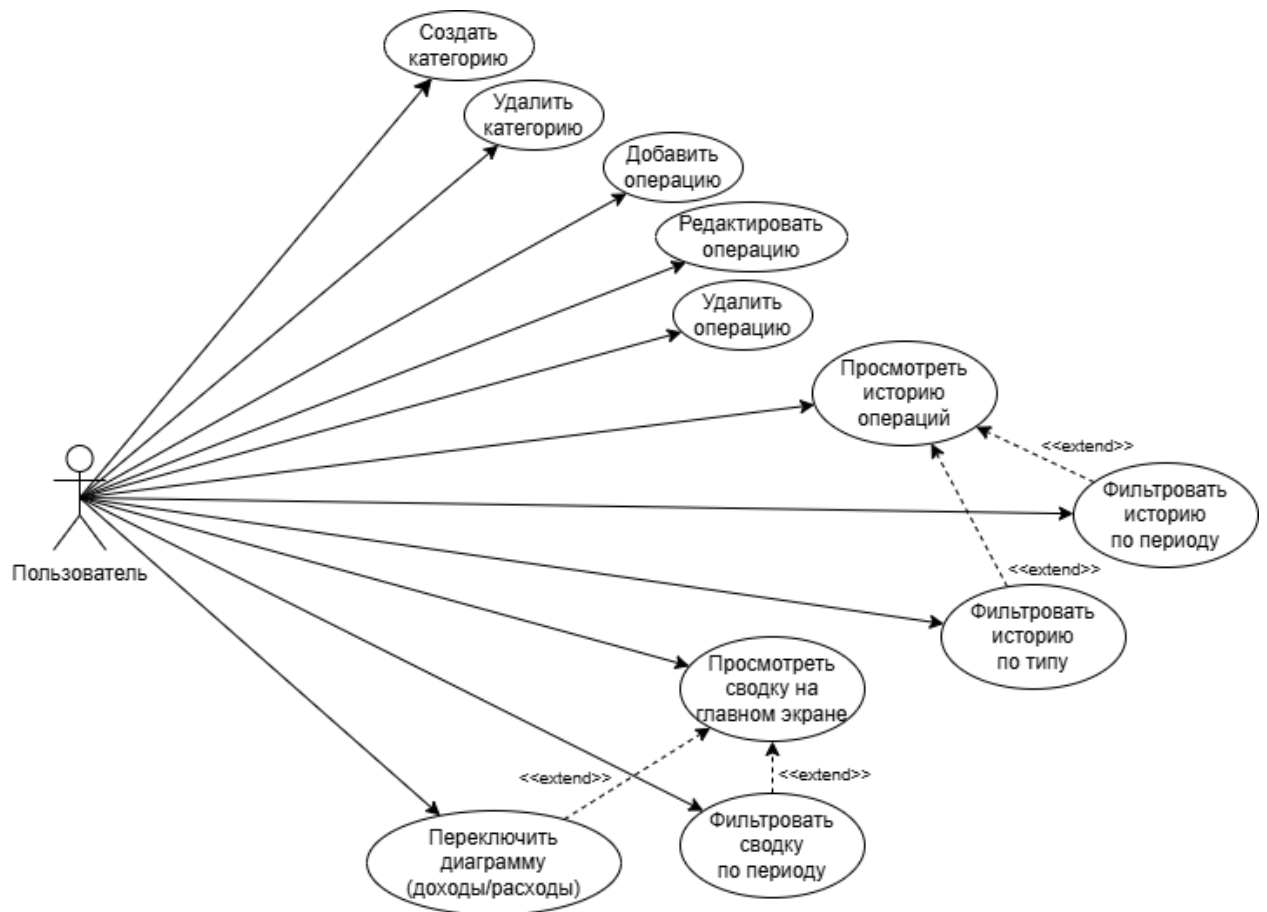


Рисунок 2.1 — Диаграмма прецедентов разрабатываемого проекта

2.1.2. Диаграмма последовательности

Диаграмма включает 5 объектов – Пользователь, Экран приложения (UI), Менеджер состояния (ViewModel), Хранилище данных (Repository), База данных SQLite.

Результат создания диаграммы представлен на Рисунок 2.2.

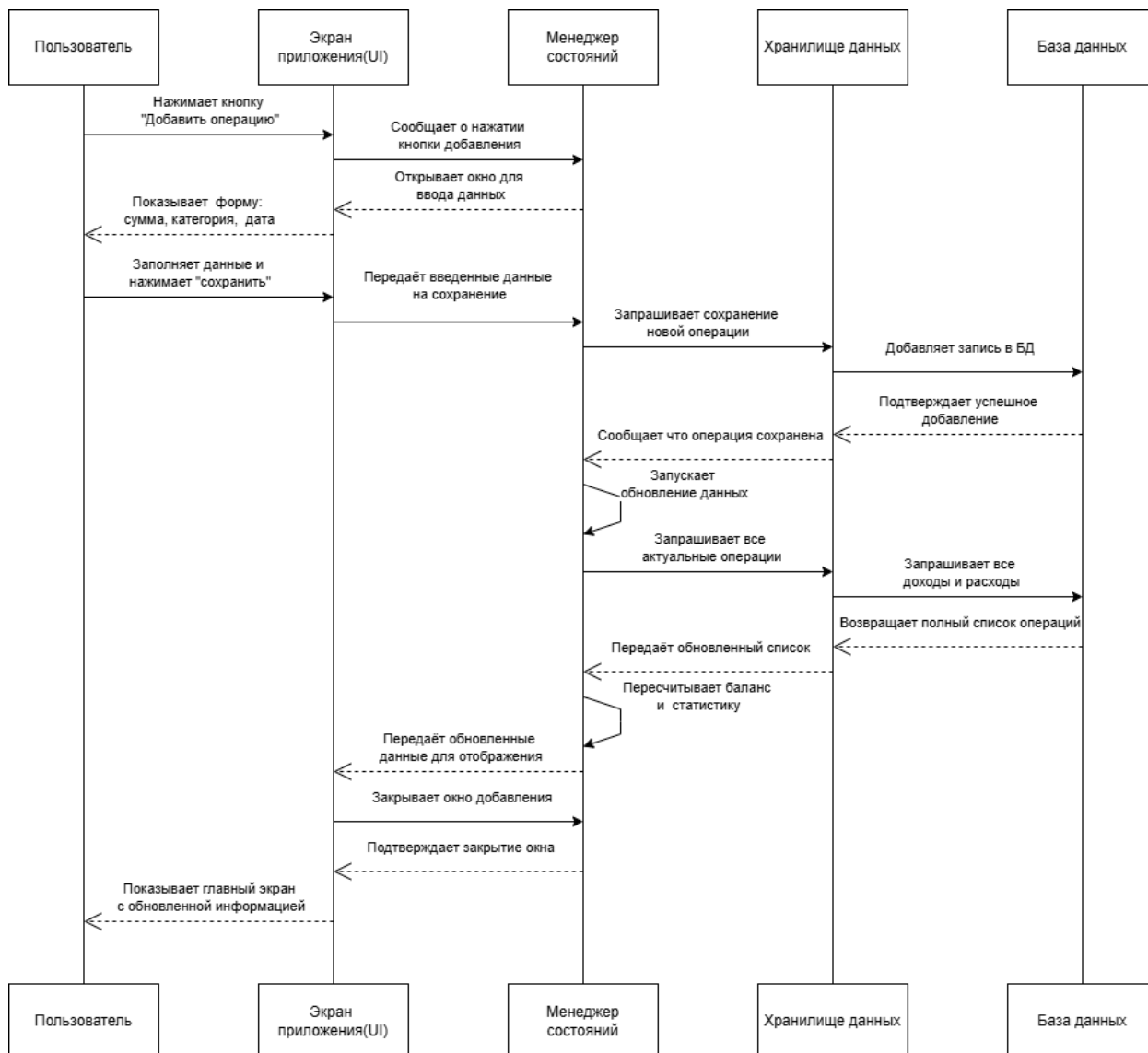


Рисунок 2.2 — Диаграмма последовательности разрабатываемого проекта

2.2. Процесс работы в системе

Для детализации поведения системы и взаимодействия пользователя с ней на уровне конкретных шагов была разработана диаграмма бизнес-процесса (BPMN). В качестве примера был выбран ключевой сценарий использования — «Добавление новой финансовой операции», который наглядно демонстрирует путь пользователя и логику обработки данных в приложении. (Рисунок 2.3)

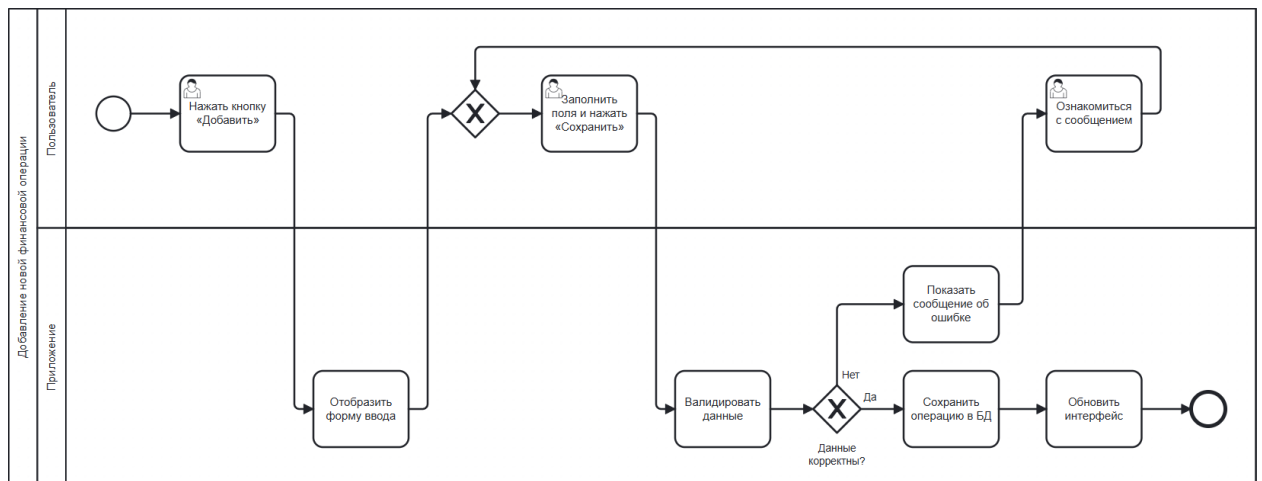


Рисунок 2.3 — Бизнес-процесс "Добавление новой финансовой операции"

2.3. Структурные диаграммы

2.3.1. Диаграмма классов

Диаграмма классов, представленная на Рисунок 2.4, отображает основные сущности приложения и их взаимоотношения, являясь основой для будущей реализации.

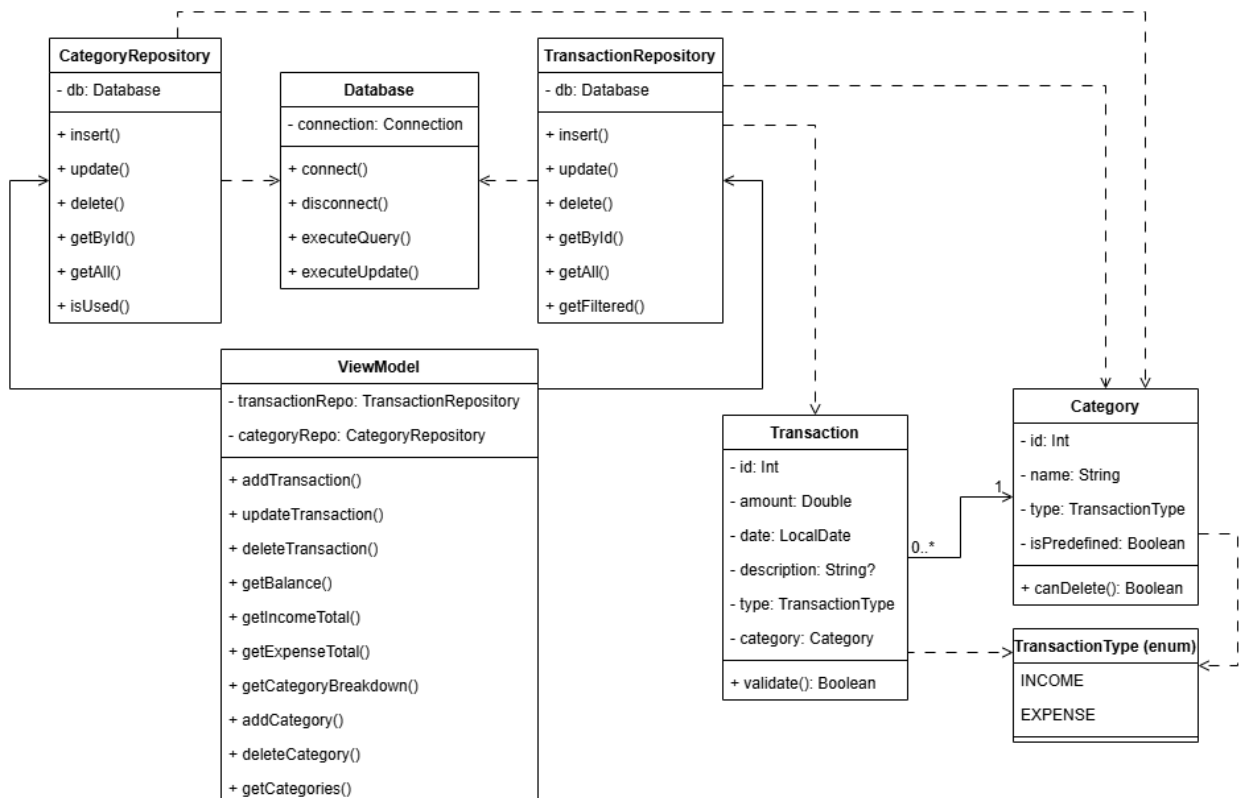


Рисунок 2.4 — Диаграмма классов

2.3.2. Диаграмма объектов

Для иллюстрации работы системы в конкретный момент времени была построена диаграмма объектов (Рисунок 2.5). Она показывает экземпляры классов, созданные в процессе выполнения пользовательского сценария, и значения их атрибутов.

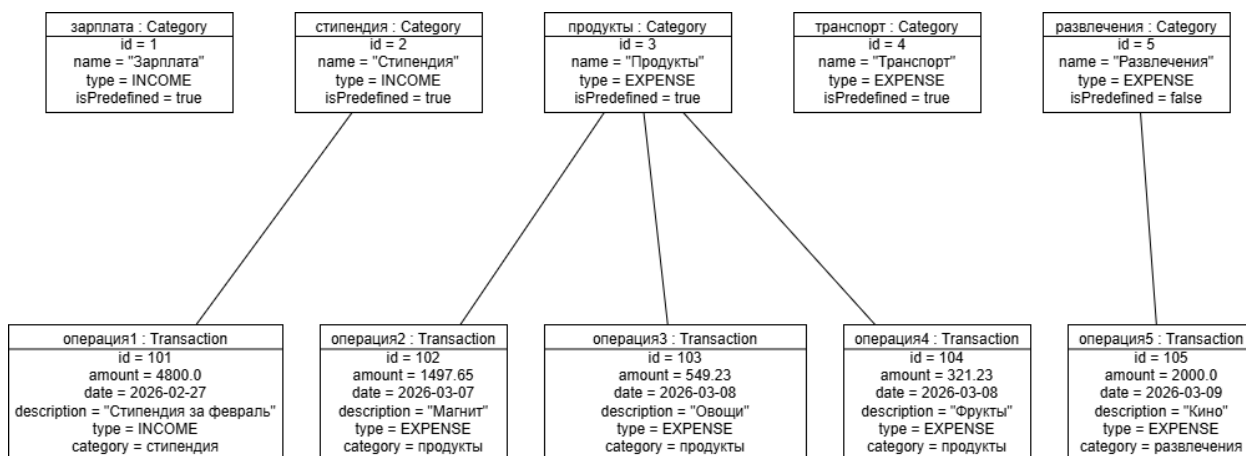


Рисунок 2.5 — Диаграмма объектов

2.4. Информационное взаимодействие компонентов системы

2.4.1. Текстовое описание

Взаимодействие начинается с того, что Пользователь системы инициирует работу, отправляя два основных типа запросов: Команды управления (на изменение данных) и Запросы на просмотр (на получение информации). Эти запросы могут сопровождаться Параметрами фильтрации для уточнения выборки. Все они поступают в ядро системы — Систему управления личными финансами, которая обрабатывает их и, при необходимости, транслирует в Запросы к БД для взаимодействия с базой данных.

В ответ на пользовательские действия система формирует Результаты запросов, которые затем преобразуются в Отображение данных для пользователя. Если в процессе обработки возникают ошибки (например, при некорректном Запросе на управление операциями или категориями), система генерирует Сообщения об ошибках, которые также выводятся пользователю.

Таким образом, замкнутый цикл "запрос — обработка — ответ" обеспечивает актуальность информации на экране.

Более детально взаимодействие раскрывается в специализированных сценариях. Так, Запрос на аналитику инициирует цепочку вычислений: сначала происходит Расчет финансовой сводки, на основе которого подготавливаются Данные для визуализации, и только потом выполняется Построение диаграмм. Результатом этого процесса является готовая Финансовая сводка и визуальные элементы, отображаемые пользователю.

Управление данными также структурировано по модулям. При выполнении Запроса на управление операциями система позволяет выполнять Добавление, Редактирование или Удаление операции, манипулируя соответствующими Данными операций. Аналогично, Запрос на управление категориями дает возможность Добавлять, Удалять или Просматривать категории, работая с Данными категорий. Ключевым результатом фильтрации и просмотра является формирование Отфильтрованного списка операций, который служит основой как для аналитики, так и для непосредственного ознакомления пользователя с историей транзакций.

2.4.2. Диаграмма DFD

На основании текстового описания информационного взаимодействия компонентов системы была построена диаграмма в нотации DFD. На рисунках ниже представлен верхний уровень диаграммы, декомпозиция верхнего уровня и функций внутри нее («Управление интерфейсом», «Управление операциями», «Управление категориями», «Аналитика и визуализация»):

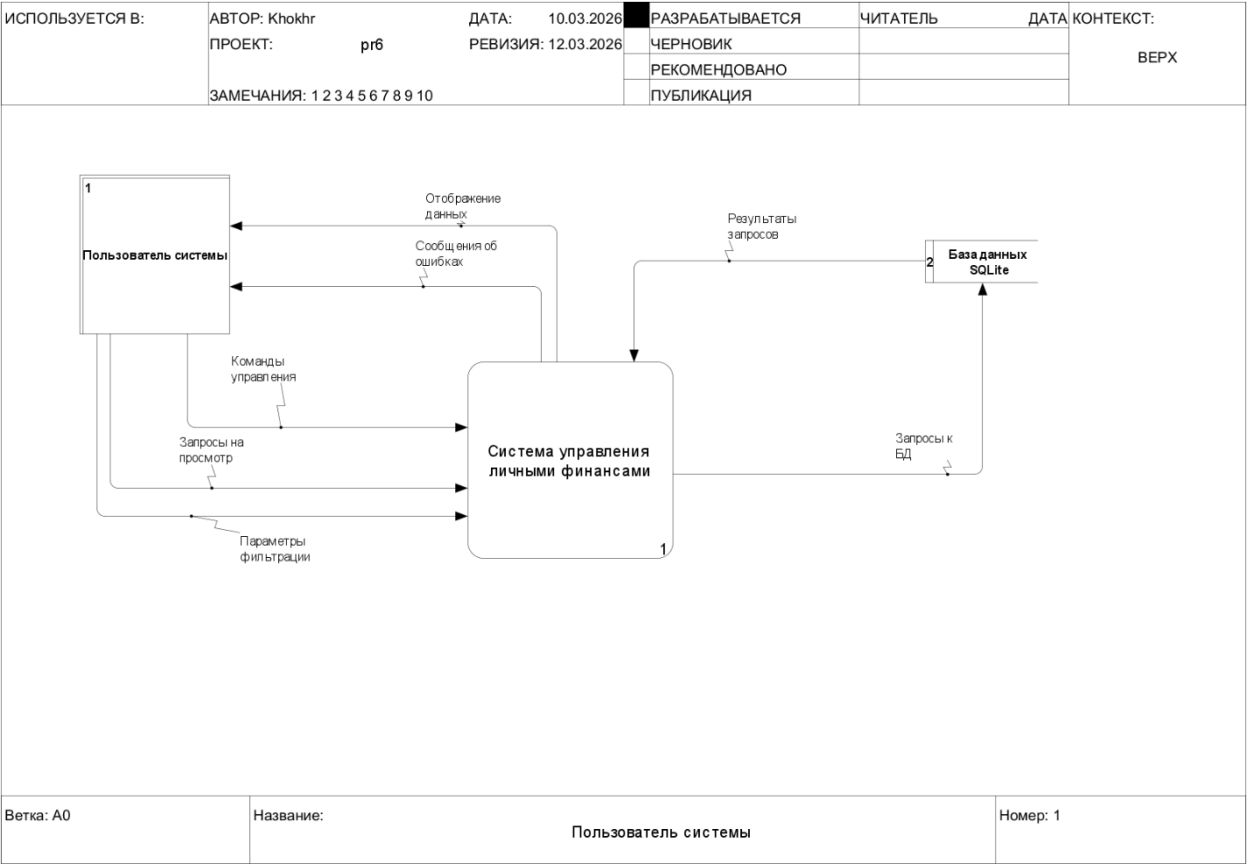


Рисунок 2.6 — Диаграмма DFD. Верхний уровень

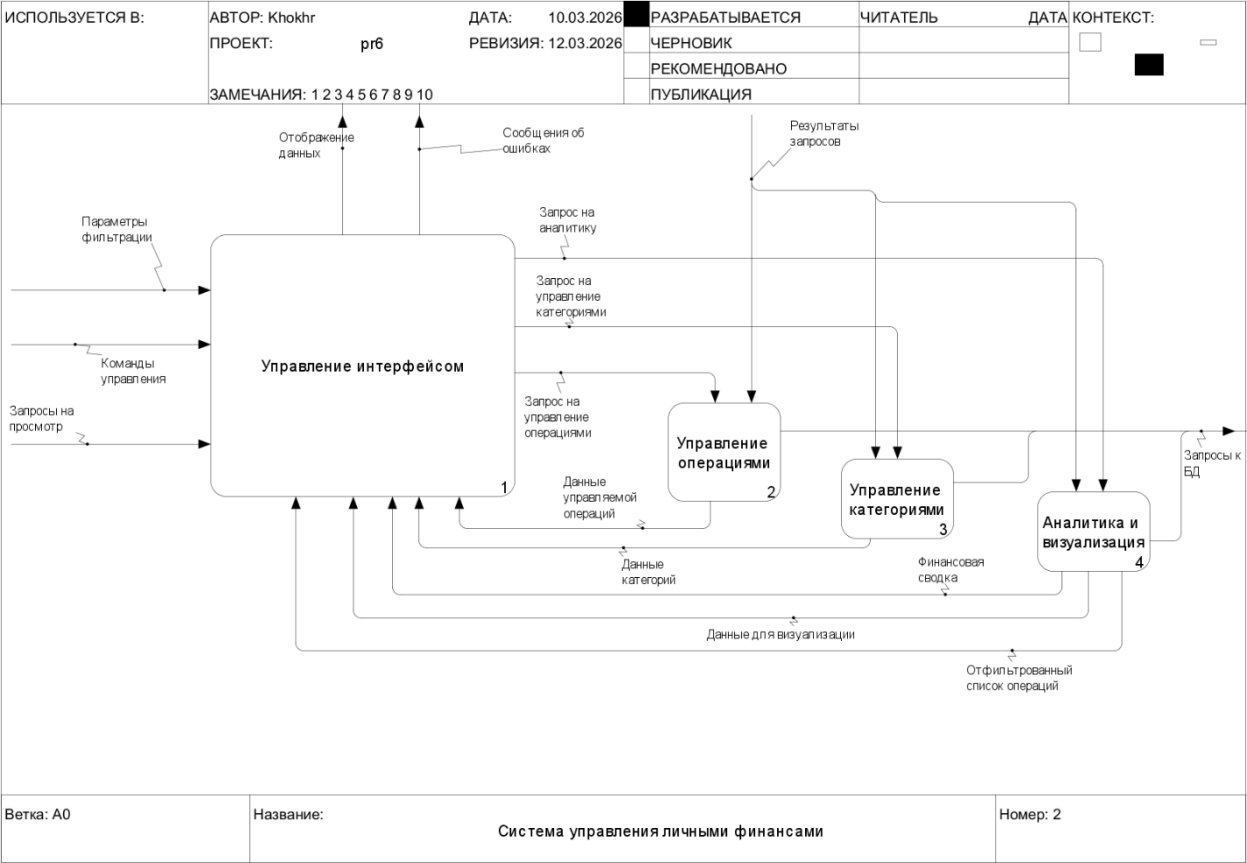


Рисунок 2.7 — Диаграмма DFD. Декомпозиция верхнего уровня

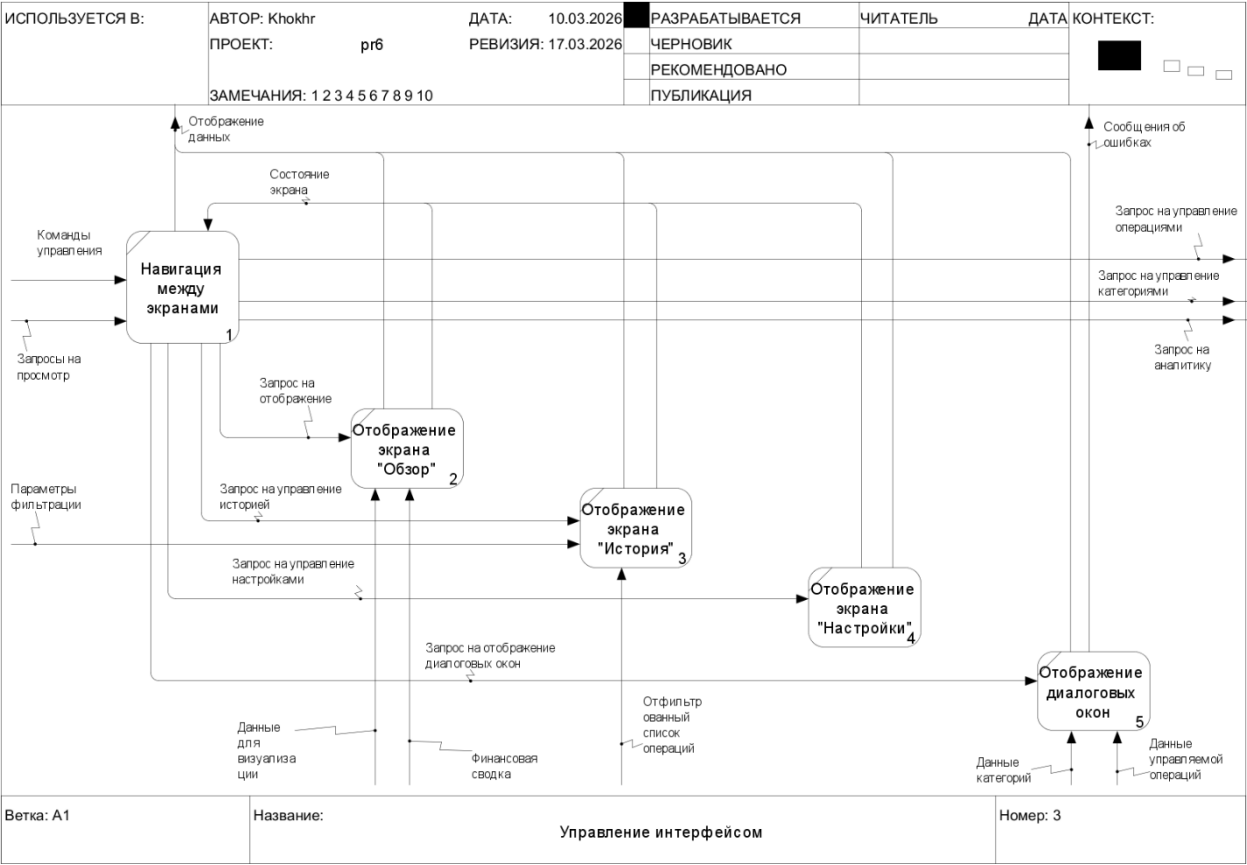


Рисунок 2.8 — Диаграмма DFD. Декомпозиция "Управление интерфейсом"



Рисунок 2.9 — Диаграмма DFD. Декомпозиция "Управление операциями"

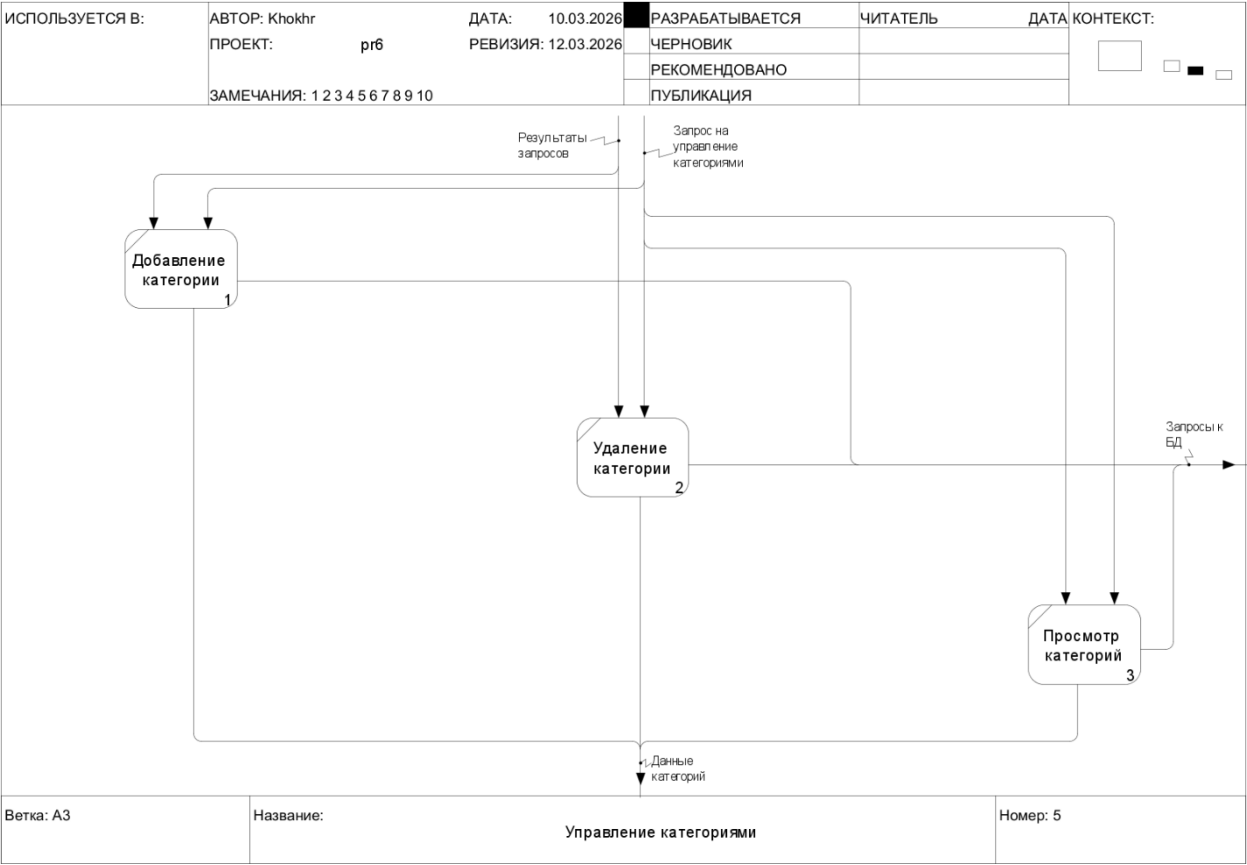


Рисунок 2.10 — Диаграмма DFD. Декомпозиция "Управление категориями"

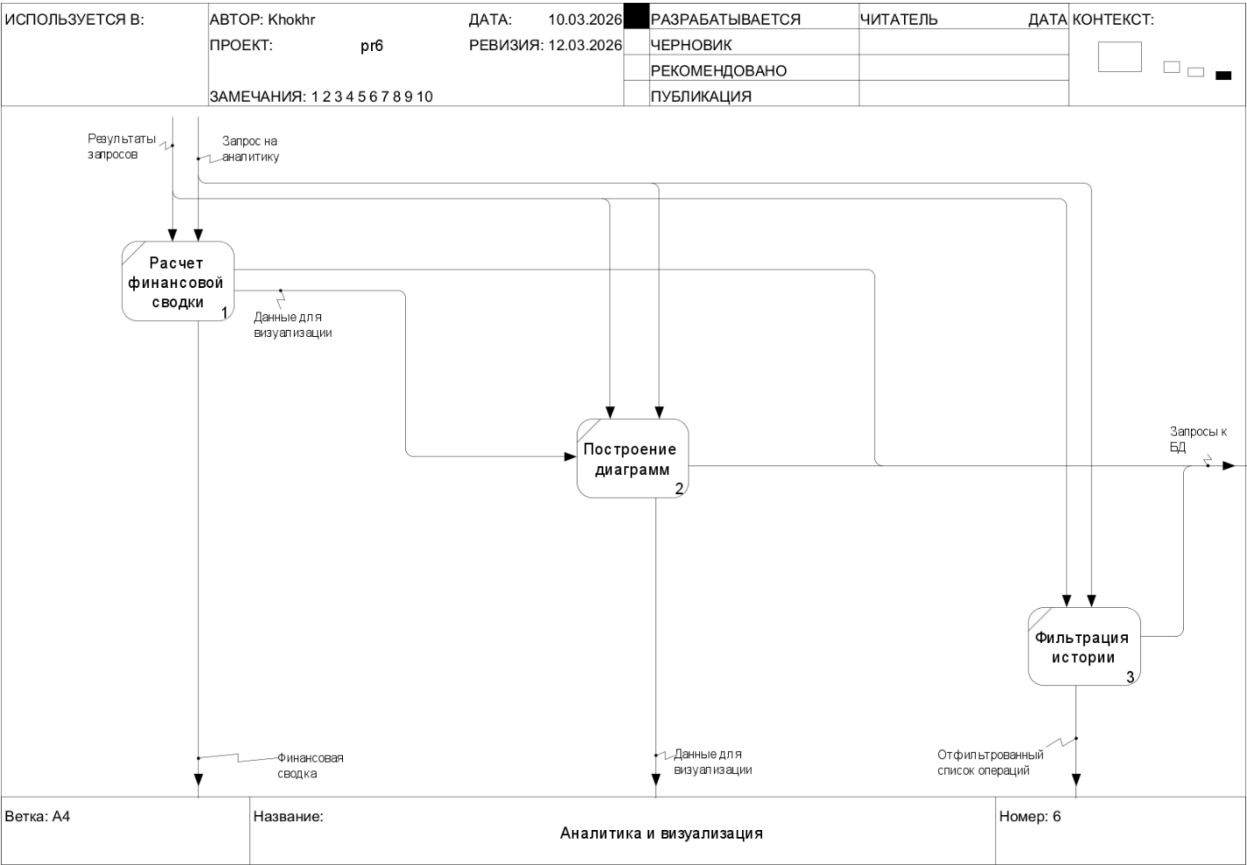


Рисунок 2.11 — Диаграмма DFD. Декомпозиция "Аналитика и визуализация"

2.5. Модель Базы данных

В качестве системы управления базами данных для офлайн-хранения информации выбрана SQLite — легковесная, встраиваемая реляционная база данных, идеально подходящая для десктопных приложений, работающих с локальными данными.

Логическая модель данных (Рисунок 2.12) состоит из четырех основных таблиц, связанных отношением "один ко многим":

1. Income_categories (Категории доходов):
 - id_income_category (INTEGER, PRIMARY KEY) — уникальный идентификатор категории.
 - income_category_name (TEXT, NOT NULL, UNIQUE) — название категории (например, "Зарплата", "Стипендия").
2. expenses_categories (Категории расходов):
 - id_expenses_category (INTEGER, PRIMARY KEY) — уникальный идентификатор категории.
 - expenses_category_name (TEXT, NOT NULL, UNIQUE) — название категории (например, "Еда", "Транспорт").
3. Income_transactions (Транзакции доходов):
 - id_income_transaction (INTEGER, PRIMARY KEY) — уникальный идентификатор транзакции.
 - income_transaction_name (TEXT) — описание транзакции (может быть NULL).
 - income_transaction_sum (REAL, NOT NULL) — сумма дохода.
 - id_income_category (INTEGER, NOT NULL, FOREIGN KEY) — ссылка на идентификатор категории дохода из таблицы Income_categories. Обеспечивает целостность данных на уровне БД.
 - income_transaction_date (DATE, NOT NULL) — дата операции в формате ГГГГ-ММ-ДД.

4. expenses_transactions (Транзакции расходов):
- id_expenses_transaction (INTEGER, PRIMARY KEY) — уникальный идентификатор транзакции.
 - expenses_transaction_name (TEXT) — описание транзакции (может быть NULL).
 - expenses_transaction_sum (REAL, NOT NULL) — сумма расхода.
 - id_expenses_category (INTEGER, NOT NULL, FOREIGN KEY) — ссылка на идентификатор категории расхода из таблицы expenses_categories.
 - expenses_transaction_date (DATE, NOT NULL) — дата операции.



Рисунок 2.12 — Логическая модель Базы данных

2.6. Архитектура приложения

Для проекта "Finlytics" выбрана чистая многослойная архитектура (Layered Architecture), реализованная в рамках паттерна MVVM (Model-View-

ViewModel) на клиентской стороне. Это классический и наиболее подходящий выбор для десктопного офлайн-приложения.

2.6.1. Обоснование выбора

1. Соответствие типу приложения: Разрабатываемое приложение — десктопное, офлайн, однопользовательское. Ему не требуется сложная распределенная архитектура (как SOA или микросервисы) или разделение на физические уровни (как трехуровневая архитектура). Многослойная архитектура идеально подходит для таких монолитных приложений, обеспечивая логическое разделение ответственности без излишнего усложнения инфраструктуры.

2. Принцип разделения ответственности (SoC): Архитектура четко разделяет приложение на слои: UI (View) отвечает только за отображение, ViewModel — за состояние и логику представления, а Model (Репозиторий и источник данных) — за бизнес-логику и данные. Это делает код понятным, тестируемым и легким в сопровождении.

3. Гибкость и расширяемость: Паттерн MVVM и многослойность обеспечивают гибкость. Мы можем изменить способ хранения данных, модифицируя только слой Repository и источники данных, не трогая при этом ViewModel и UI. Аналогично, изменение интерфейса не влияет на бизнес-логику.

4. Поддержка реактивности: Kotlin Flows, используемые в ViewModel, идеально вписываются в эту архитектуру, позволяя UI автоматически реагировать на изменения в состоянии (State).

5. Простота для команды: Архитектура понятна всем разработчикам, использующим Kotlin, и не требует изучения сложных распределенных концепций.

Таблица 2.1 – Программные решения и обоснования их выбора

Компонент архитектуры	Выбранное решение	Обоснование
Язык	Kotlin	Современный, лаконичный и безопасный язык

реализации		для платформы JVM. Идеально подходит для разработки десктопных приложений с Compose для Desktop. Обеспечивает отличную совместимость с существующими Java-библиотеками.
UI-фреймворк (Слой View)	Jetpack Compose for Desktop	Современный декларативный фреймворк для создания нативных UI. Позволяет писать меньше кода, он более интуитивен и легче поддерживается по сравнению с устаревшими инструментами типа Swing или JavaFX. Идеально интегрируется с реактивной моделью MVVM через Kotlin Flow.
Управление состоянием	Kotlin Coroutines & Flow (ViewModel)	Flow обеспечивает реактивный поток данных. ViewModel подписывается на изменения в Repository (которые также могут быть представлены как Flow) и предоставляет StateFlow для UI. Coroutines используются для асинхронных операций с БД, не блокируя главный поток UI.
Слой данных (Repository)	Собственный класс FinanceRepository	Выступает единой точкой входа для всех данных. Абстрагирует источник данных (БД) от вышележащих слоев.
Слой источника данных	SQLite	Легковесная, встраиваемая реляционная база данных, идеально подходящая для локального хранения структурированных данных на одном устройстве. Не требует отдельного сервера и настройки, что критически важно для офлайн-приложения.
Работа с БД	Прямое использование JDBC (встроенные средства Kotlin/Java)	Для такого небольшого проекта использование ORM было бы избыточным. Прямые SQL-запросы с использованием PreparedStatement дают полный контроль над производительностью и безопасностью (защита от SQL-инъекций).
Сборка и зависимости	Gradle (Kotlin DSL)	Стандарт де-факто для сборки Kotlin-проектов. Kotlin DSL обеспечивает типовую безопасность и лучшую поддержку IDE по сравнению с Groovy DSL.
Логирование	Стандартный println (с настройкой UTF-8 в LoggingConfig)	Для отладки и мониторинга состояния приложения на этапе разработки. Простота использования достаточна для нужд проекта.
Визуализация (диаграммы)	Кастомный компонент на Canvas (PieChart.kt)	Compose для Desktop предоставляет мощный Canvas API, который позволяет полностью контролировать отрисовку. Создание собственной диаграммы дает гибкость в дизайне и избавляет от необходимости подключать тяжелые сторонние библиотеки.
Векторная графика	Собственный набор ImageVector (FinlyticsIconPack)	Использование встроенной в Compose возможности работы с векторной графикой. Иконки хранятся в виде кода (Kotlin-файлов), что делает их неотъемлемой частью приложения, упрощает версионирование и не

		требует внешних ресурсов.
Контроль версий	Git (платформа GitHub)	Обеспечивает совместную работу команды, отслеживание истории изменений и является стандартом в индустрии.

2.6.2. Диаграмма архитектуры

На Рисунок 2.13 представлена диаграмма, отражающая спроектированную и реализованную архитектуру приложения "Finlytics". Диаграмма иллюстрирует многослойную структуру с паттерном MVVM.

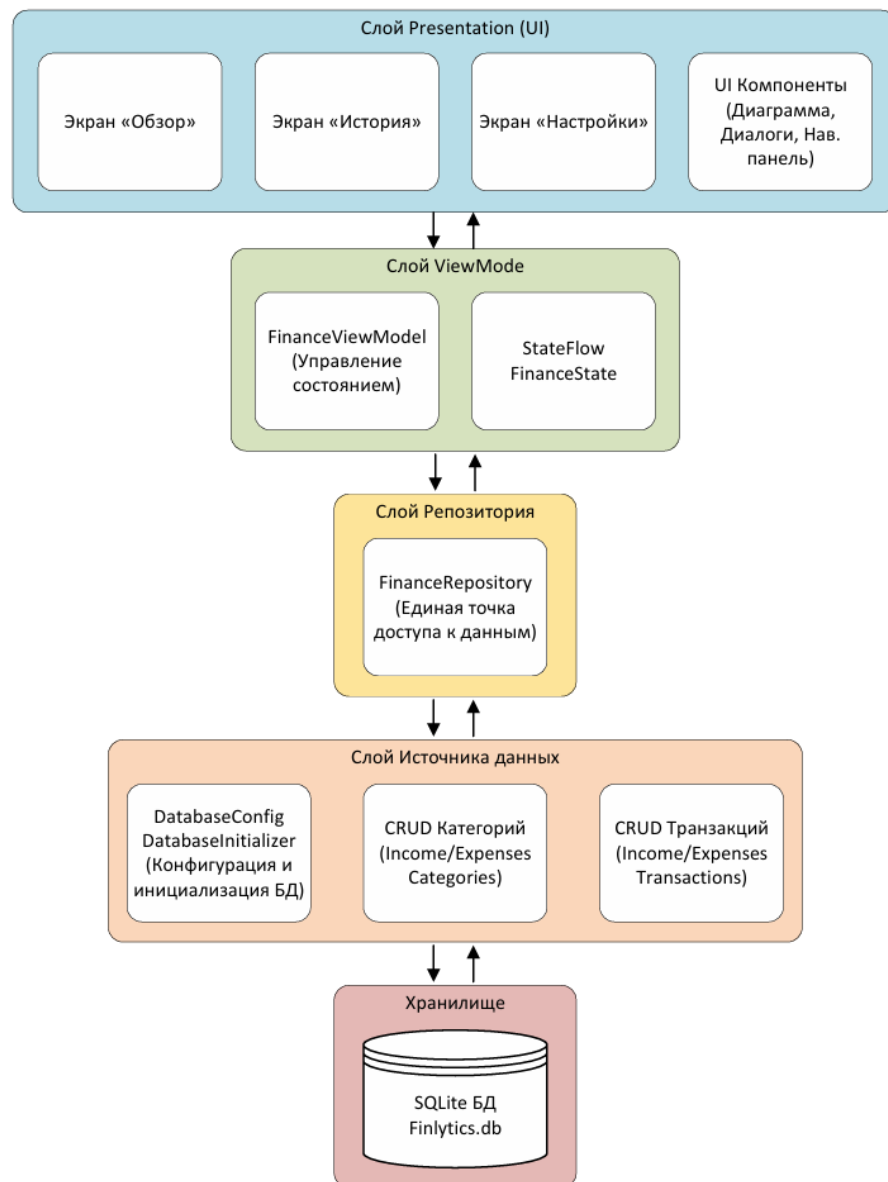


Рисунок 2.13 — Диаграмма архитектуры приложения «Finlytics»